Variables, Input & Output

上海纽约大学
NYU SHANGHAI

CSCI-SHU 11
Fall 2025

Lecture's Outline

- Variables
  - assignment
  - naming rules
- User input
  - input built-in function
  - input conversion
- User output
  - Print function arguments
  - sep and end arguments
  - formatting, truncating, padding strings

Variables

Variables

- What is a variable?
  - A variable is a name that refers to a value
- To use a variable, you need to:
  - declare it and
  - assign it a value
- You can then use that variable where you would use the value
- Example:

```
>>> variable_name = 'Hello World!'
>>> print(variable_name)

Hello World!
```

Variables

- How do variables actually work ?

```
some_variable_name = "a value"
```

- This is an assignment statement
  - it binds a value to a name
- The equal sign is the assignment token
  - the "operator" that we use to bind a name to a value
  - name on left (must respect some rules)
  - value on right (can be string, or numeric value, or . . .)

Variables and Memory Allocation

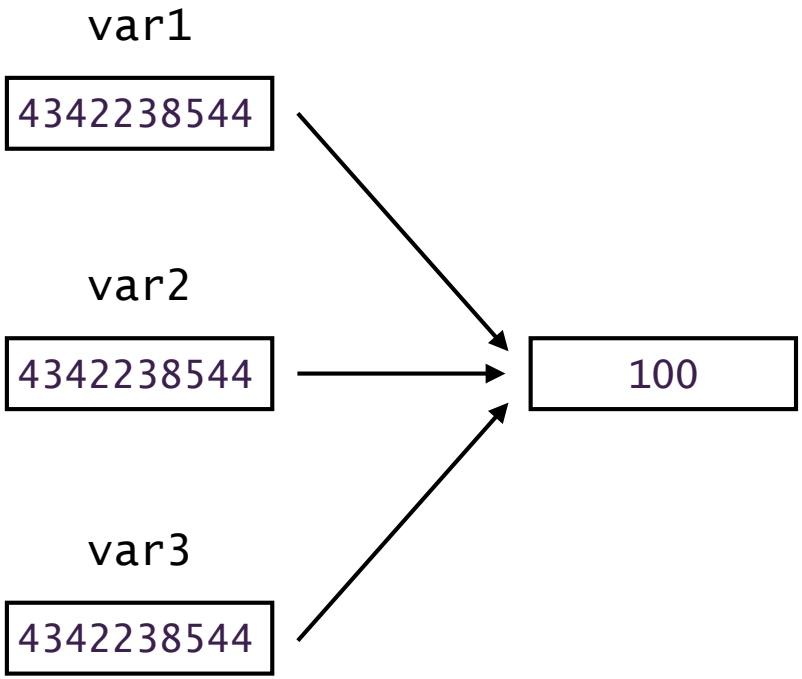
- A variable is a value stored in memory
  - its name is a sudo for the address of the memory block allocated
    - the cake is a value
    - the rack/compartment is a memory block where the value is stored
- Built-in function *id* shows the address in memory of a variable

```
>>> var1 = 100
>>> var2 = var1
>>> var3 = 100
>>> id(var1)
4342238544
>>> id(var2)
4342238544
>>>id(var3)
4342238544
```

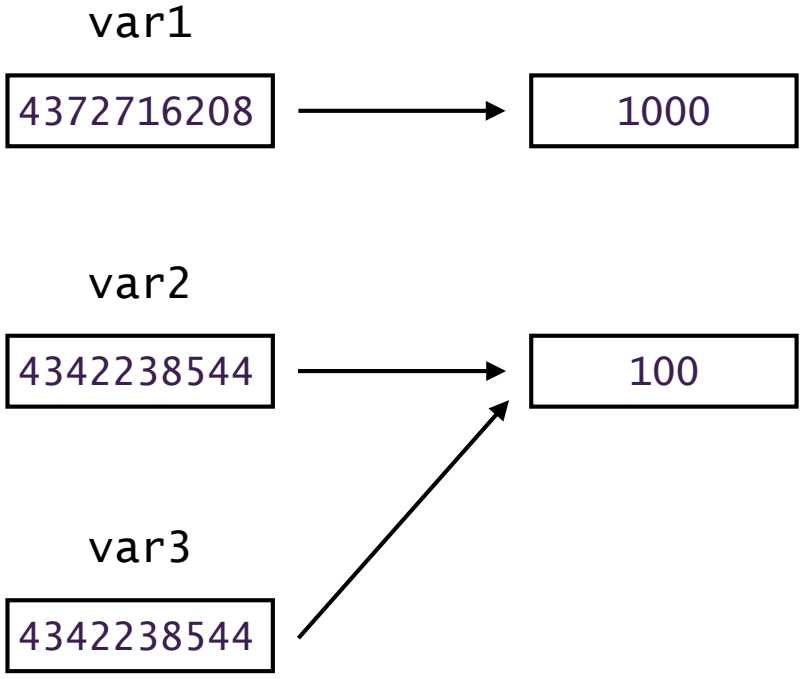
- The size of the memory blocks allocated for a variable depends on the type
  - int variables occupy 32 bits in memory

Variables and Memory Allocation

```
>>> var1 = 100
>>> var2 = var1
>>> var3 = 100
```




```
>>> var1 = 100
>>> var2 = var1
>>> var3 = 100
>>> var1 = 1000
```



Variables

- When do use variables?
  - when you want your program to remember a value
  - if you have a value that will vary
- Example:
  - the input from the user once prompted
  - a counter (number of repetitions)
  - an accumulator (average grade of a student)
  - …

Variables

Interactive Shell

- Whenever you type something in the shell, it will always return a value
  - a value returns a value
  - a variable returns a value
  - a function can return a value
  - if a function does not return a value, the resulting value will be:

    `None (NoneType)`
- How to distinguish something printed vs a returned value?
  - returning a string shows the string in quotes,
  - while printing displays the string without quotes

```
>>> print("Hello")
Hello
>>> print("2")
2
```

```
>>> "Hello"
'Hello'
>>> print(2)
2
```

```
>>> 2
2
```

Variables

Not defined variables

- Variables that are not defined yet

```
>>> variable_name
variable_name
Traceback (most recent call last):
  File "<pyshell#39>", line 1, in <module>
    variable_name
NameError: name 'variable_name' is not defined

>>> variable_name = 2
>>> variable_name
2
```

- Before using any variable, **it needs to be assigned!**

Variables

Reassignment

- You can reassign or rebind a variable

```
>>> variable_name = 2
>>> variable_name
2
>>> variable_name = "2"
>>> variable_name
'2'
```

Variables

Variable Naming Rules

- The name for a variable
  - can be as long as you want
  - cannot contain spaces
  - consists of alphanumeric (numbers and letters) and the underscore first character
    - first character cannot be a number (only a letter or underscore)
  - is case sensitive
    - foo and Foo are 2 different variables
  - cannot be a keyword (print, input, if, then, elif, else, while, for, …)

Variables

Variable Naming Rules

- Which of the following are valid variable names in Python?

`_foo`	`foo bar`
`$foo`	`Fo0`
`foo`	`foobar123`
`3foo`	`#foo`

Variables

Variable Naming Rules

- Which of the following are valid variable names in Python?

`_foo`	`foo bar SyntaxError: invalid syntax. Perhaps you forgot a comma?`
`$foo SyntaxError: invalid syntax`	`FoO`
`foo`	`foobar123`

`3foo SyntaxError: invalid decimal literal`	`#foo`
	`Traceback (most recent call last):`
	`  File "<pyshell#62>", line 1, in <module>`
	`  foo`
	`NameError: name 'foo' is not defined.`

Variables

- Create a variable called *exclaim*
- Set it equal to "!!!" and print out the variable

- Create 2 variables, *length* and *width*
- Set them both equal to 25 and 8 respectively
  multiply both variables
- Store the result in a variable called *area* and print out the result

Variables

- Create a variable called *exclaim*
- Set it equal to "!!!" and print out the variable

```
>>> eclaim = "!!!"
>>> print(eclaim)
    !!idle
```

- Create 2 variables, *length* and *width*
- Set them both equal to 25 and 8 respectively
  multiply both variables
- Store the result in a variable called *area* and print out the result

```
length = 25
width = 8
area = length * width
print(area)
200
```

Variables

Multiple Assignment

- It is possible to declare/assign several variables with a single line in Python

```
>>> a, b, c = 3, "Hello", 7
>>> a
    3
>>> b
    'Hello'
>>> c
    7
```

Variables

Swap variable values

- We have 2 variables *a* and *b* with values *3* and *21* respectively.
- How can we swap their values with some code?

```
# Solution 1: Using a third variable for temporary storage

a = 3
b = 21
print('a:',a)   #print value of a
print('b:',b)   #print value of b

print()         #blank line

c = b           #temp variable c
b = a
a = c
print('a:',a)   #print value of a
print('b:',b)   #print value of b
```

```
a: 3
b: 21

a: 21
b: 3
```

Variables

Swap variable values

- We have 2 variables *a* and *b* with values *3* and *21* respectively.
- How can we swap their values with some code?

```
# Solution 2: Multiple assignment with Python

a = 3
b = 21
print('a:',a)   #print value of a
print('b:',b)   #print value of b

print()         #blank line

a, b = b, a     #swap values
print('a:',a)   #print value of a
print('b:',b)   #print value of b
```

```
a: 3
b: 21

a: 21
b: 3
```

User Input

User Input

- Getting input from the Shell
  - Programs commonly need to read input typed by the user on keyboard
  - we can prompt the user through the shell (console / terminal / command prompt)
  - the user enters input through the same mechanism
- The input() function
  - There is a builtin function in Python called *input*
  - It can take one parameter - the prompt that is displayed
  - it returns a string representing the user's input (until Enter/Return key is pressed)
  - it will always return a string - even if the input is a number

User Input

```
>>> input("What is your name? ")
    What is your name? Promethee
    'Promethee'
```

wait for a user input (keyboard input until Return/Enter key is pressed)

- *input* function returns a string that you can store in a variable

```
>>> user_name = input("What is your name? ")
    What is your name? Promethee
    'Promethee'
>>> user_name
    'Promethee'
>>> print(user_name)
    Promethee
```

User Input

- Input conversion
  - The input function always return a string
  - You can use the conversion builtin functions

```
year_of_birth = int(input("Enter your year of birth)
```

- Example: Write a program that
  - prompts the user for his year of birth
  - outputs the age of the user

User Input

- Input conversion
  - The input function always return a string
  - You can use the conversion builtin functions

```
year_of_birth = int(input("Enter your year of birth)
```

- Example: Write a program that
  - prompts the user for his year of birth
  - outputs the age of the user

```
# This program calculates the age of a user

# user year of birth
year_of_birth = int(input("Enter your year of birth: "))

# current year
current_year = int(input("Enter current year: "))

# user current age
age = current_year - year_of_birth

print("You are", age,"years old")
```

Print Function

Print Function

One or More Arguments

- Print text and numeric values
  - One argument: String concatenation

```
nb_cookies = 10
print("Number of cookies: " + str(nb_cookies))
Number of cookies: 10
```

  - More than one argument:

```
nb_cookies = 10
print("Number of cookies:", nb_cookies)    # 2 parameters
Number of cookies: 10
```

    - No ending space in the string
    - No need to convert the number into a str

Print Function

The *sep* Keyword Argument

- By default the separator argument is a space:

```
>>> print('a','b')
a b
>>> print('a','b', sep=' ')
a b
```

- The separator argument can be changed with a string passed with the *sep* argument:

```
>>> print('a','b',sep='>>>')
a>>>b
>>> print('a','b', sep='"""')
a"""b
```

Print Function

The *end* Keyword Argument

- By default the end argument is a newline:

```
print('a','b', sep='*')
print('c','d', sep='?', end='\n')
print('e','f')
```

```
a*b
c?d
e f
```

- The argument separator can be changed with a string passed with the *sep* argument:

```
print(1,end=">>>")
print(2,end=">>>")
print(3,end=">>>")
print(4,end=">>>")
print(5,end=">>>")
print("Ignition!")
```

```
5>>>4>>>3>>>2>>>1>>>Ignition!
```

Format Method

String Formatting

- The format() method called on a string replaces a placeholder with a value

```
>>> "My name is {0} and I'm {1}-feet tall".format("Promethee",5.9)
    "My name is Promethee and I'm 5.9-feet tall"
```

  - {0}, {1}, … are placeholders
  - they are replaced by the parameters of the format() method

```
>>> name = "Promethee"
>>> height = 5.9
>>> "My name is {0} and I'm {1}-feet tall".format(name,height)
    "My name is Promethee and I'm 5.9-feet tall"
```

String Formatting

Padding and Aligning Strings (<, >, ^)

align left

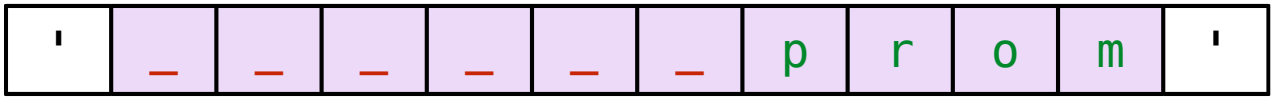
```
'{:<10}'.format('prom')
```

align right

```
'{:>10}'.format('prom')
```

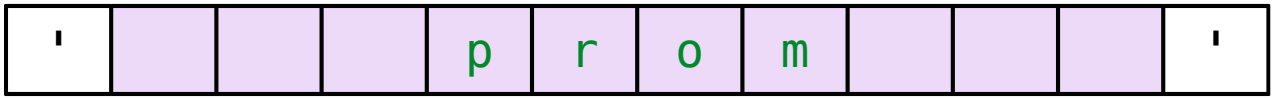
align right & pad

```
'{:_>10}'.format('prom')
```



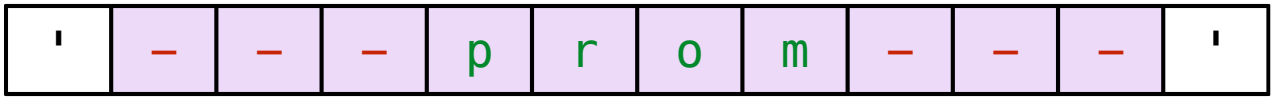
align center

```
'{:^10}'.format('prom')
```



align center & pad

```
'{:-^10}'.format('prom')
```



1 2 3 4 5 6 7 8 9 10

String Formatting

Truncating and Padding Strings

```
'{:.4}'.format('promethee')
```

'	p	r	o	m	'

```
'{:>10.5}'.format('promethee')
```

'						p	r	o	m	e	'

```
'{:_>10.6}'.format('promethee')
```

'	_	_	_	_	p	r	o	m	e	t	'

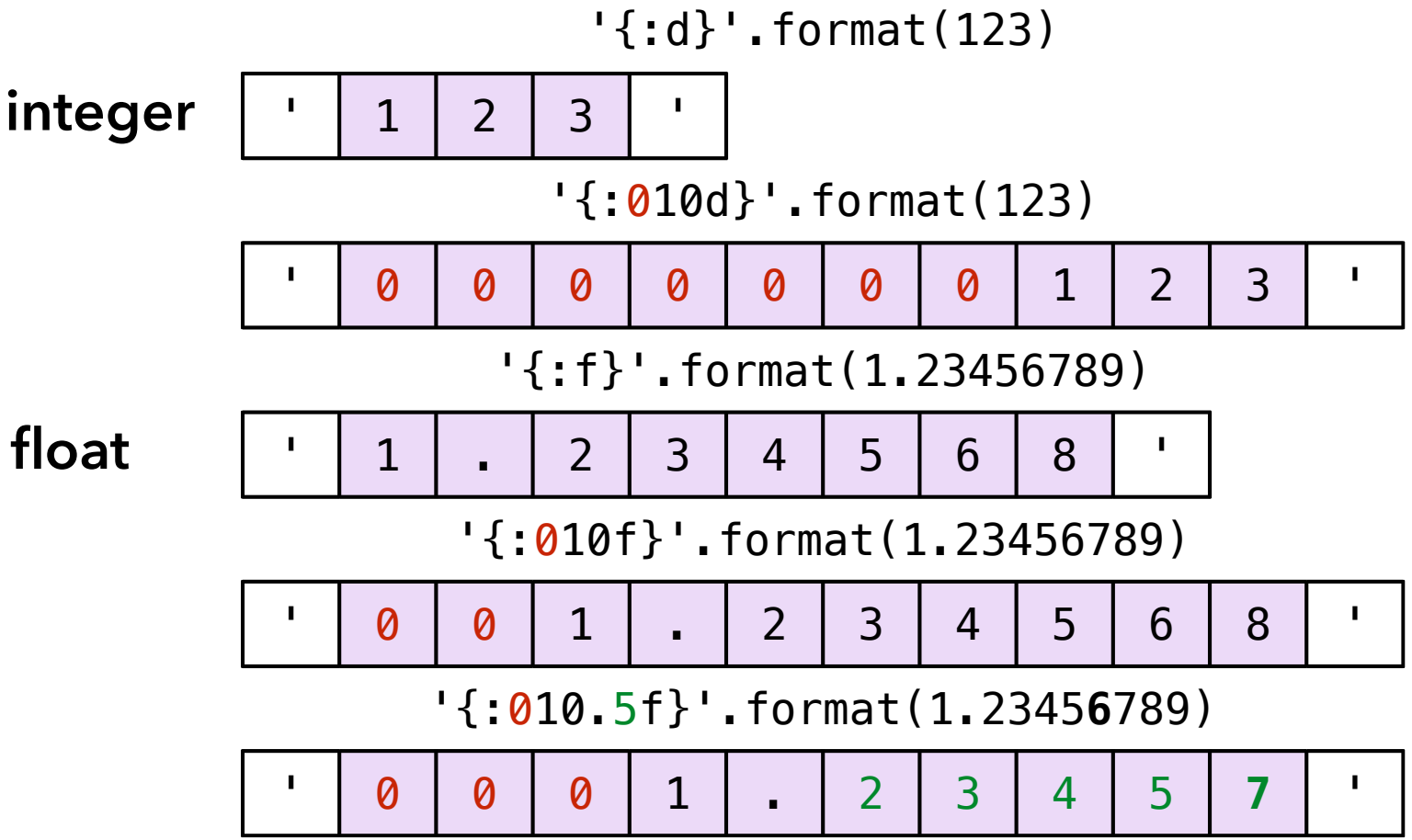
```
'{:^10.6}'.format('promethee')
```

'			p	r	o	m	e	t			'

```
'{:-^10.2}'.format('promethee')
```

'	-	-	-	-	p	r	-	-	-	-	'

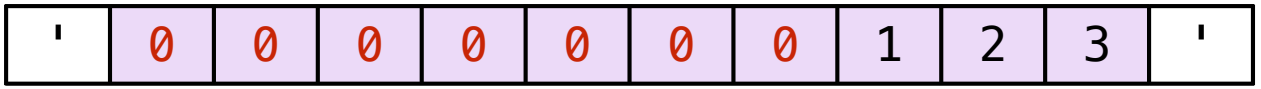
Number Formatting



String Conversion

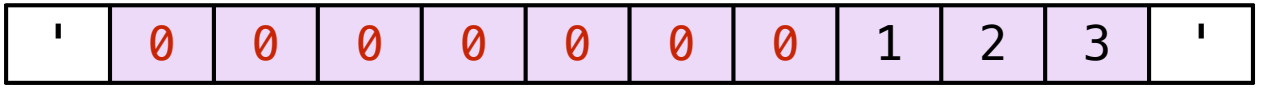
string

```
'{:0>10}'.format('123')
```



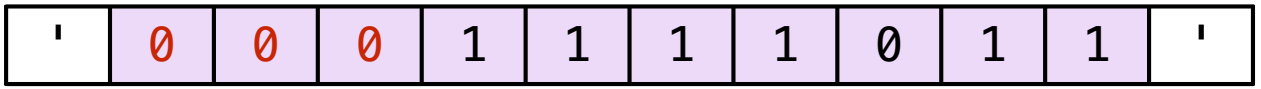
integer

```
'{:010d}'.format(123)
```



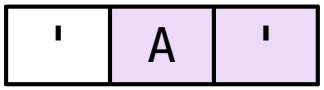
binary

```
'{:010b}'.format(123)
```



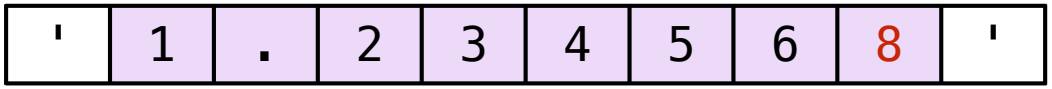
character

```
'{:c}'.format(65)
```



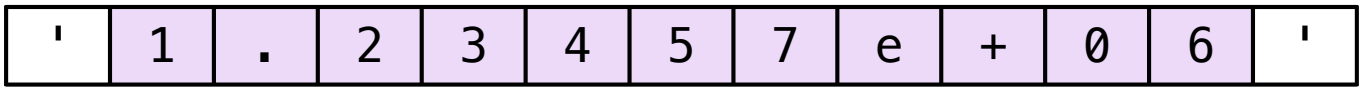
decimal

```
'{:f}'.format(1.23456789)  # 789 is rounded to 8
```



decimal

```
'{:g}'.format(1234567.89)
```



percentage

```
'{:.3%}'.format(.2666666666666)
```

Takeways

- Variables
  - Assignment: declaration and initialization in one step
- Built-in functions
  - Type conversion
  - User driven input
- Input
  - *input* Python builtin function
  - It can take one parameter - the prompt that is displayed
  - it returns a string representing the user's input
- Output & string formatting
  - the print function can have many parameters
  - the last parameters include the sep and end keyword
  - string formatting allows for precise control over the format of the output

Readings & Assignments

- Preparation for Lecture 04
  - Read chapter 3 in the textbook
  - Watch self-paced modules 3 (use your NYU login/password):
    - https://stream.nyu.edu/playlist/dedicated/306991832/1_gm6t0b7p/
- Assignments (deadline: 09/16 09:45am):
  - Quizz 01a
  - Quizz 01b